Teoria współbieżności

Sprawozdanie z laboratorium 3

Adrian Mikoda

24.10.2026 r.

Spis treści

1. Dane techniczne 2
2. Treści zadań 2
   2.1. Zadanie 1 2
   2.2. Zadanie 2 2
3. Realizacja ćwiczenia 2
   3.1. Zadanie 1 – wykorzystanie metod `wait()` i `notify()` 2
   3.2. Zadanie 1 – wykorzystanie semaforów 5
   3.3. Zadanie 2 – przetwarzanie potokowe 7
4. Wnioski 9

1. Dane techniczne

- System operacyjny: `Windows 11 Home`
- Procesor: `AMD Ryzen 5 7530U (2.0 - 4.5 GHz)`
- Pamięć RAM: `16GB LPDDR4 (2133 MHz)`
- SDK: `Microsoft OpenJDK 25.0.4`

2. Treści zadań

2.1. Zadanie 1

Dany jest bufor, do którego producent może wkładać dane, a konsument pobierać. Napisać program, który zorganizuje takie działanie producenta i konsumenta, w którym zapewniona będzie własność bezpieczeństwa i żywotności. Zrealizować program:

1. przy pomocy metod `wait()`/`notify()`:
   1. dla przypadku 1 producent/1 konsument,
   2. dla przypadku n_1 producentów/n_2 konsumentów ($n_1 > n_2$, $n_1 = n_2$, $n_1 < n_2$),
   3. wprowadzić wywołanie metody `sleep()` i wykonać pomiary, obserwując zachowanie producentów/konsumentów,
2. przy pomocy operacji `P()`/`V()` dla semafora:
   1. $n_1 = n_2 = 1$,
   2. $n_1 > 1, n_2 > 1$.

2.2. Zadanie 2

Zaimplementować rozwiązanie przetwarzania potokowego (przykładowe założenia: bufor rozmiaru 100, 1 producent, 1 konsument, 5 uszeregowanych procesów przetwarzających). Od czego zależy prędkość obróbki w tym systemie? Rozwiązanie za pomocą semaforów lub monitorów. W implementacji nie jest dozwolone korzystanie z własnych kolejek FIFO, należy używać tylko mechanizmu monitorów lub semaforów.

3. Realizacja ćwiczenia

3.1. Zadanie 1 – wykorzystanie metod `wait()` i `notify()`

Klasy `Producer`, `Consumer`, `Buffer` oraz `PCWaitNotify` zaimplementowano w pliku `PCWaitNotify.java` (fragment kodu 1). Bufor jest cykliczną tablicą o rozmiarze 100. Przechowuje on liczbę zajętych miejsc `size` oraz indeks najstarszego elementu `firstIndex`. Nowy element trafia na pozycję `(firstIndex + size) % bufferSize`, a pobierany jest element spod `firstIndex`, który następnie przesuwa się cyklicznie.

Metody `put()` i `get()` są `synchronized`. Gdy bufor jest pełny (lub pusty), producent (konsument) wypisuje komunikat o oczekiwaniu i wywołuje `wait()` w pętli `while`. Po wykonaniu operacji wątek wywołuje `notifyAll()`.

Każdy producent i konsument ma identyfikator widoczny w komunikatach, wykonuje zadaną liczbę iteracji i po każdej zasypia na 100 ms za pomocą `sleep()`. Wątkami

zarządza `ExecutorService` z pulą o rozmiarze $n_1 + n_2$. Program uruchamiano dla konfiguracji $n_1 = n_2 = 1$, $n_1 = n_2 = 2$, $n_1 > n_2$ ($n_1 = 3, n_2 = 1$) oraz $n_1 < n_2$ ($n_1 = 1$, $n_2 = 3$). Liczby iteracji dobrano tak, aby $n_1 \cdot \text{liczba produkcji} = n_2 \cdot \text{liczba konsumpcji}$.

Zaobserwowane zachowanie:

- $n_1 = n_2 = 1$ lub $n_1 = n_2 > 1$ – producenci i konsumenci działają na przemian, bufor pozostaje prawie pusty,
- $n_1 > n_2$ – bufor szybko się zapełnia i czekają producenci,
- $n_1 < n_2$ – bufor jest najczęściej pusty i czekają konsumenci.

```
1  class Producer implements Runnable {
2      private final Buffer buffer;
3      private final int id, iterations;
4
5      public Producer(int id, int iterations, Buffer buffer) {
6          this.id = id;
7          this.iterations = iterations;
8          this.buffer = buffer;
9      }
10
11     public void run() {
12         for (int i = 0; i < iterations; ++i) {
13             buffer.put(id, i);
14             try { Thread.sleep(100); }
15             catch (InterruptedException e) { Thread.currentThread().interrupt(); }
16         }
17     }
18 }
19
20 class Consumer implements Runnable {
21     private final Buffer buffer;
22     private final int id, iterations;
23
24     public Consumer(int id, int iterations, Buffer buffer) {
25         this.id = id;
26         this.iterations = iterations;
27         this.buffer = buffer;
28     }
29
30     public void run() {
31         for (int i = 0; i < iterations; ++i) {
32             try { Thread.sleep(100); }
33             catch (InterruptedException e) { Thread.currentThread().interrupt(); }
34             buffer.get(id);
35         }
36     }
37 }
38
39 class Buffer {
40     protected final int[] buffer;
41     protected final int bufferSize;
42     protected int size, firstIndex;
43
44     public Buffer(int n) {
45         this.bufferSize = n;
46         this.buffer = new int[n];
47         this.size = 0;
```

```
48          this.firstIndex = 0;
49      }
50
51      public synchronized void put(int id, int n) {
52          while (size == bufferSize) {
53              System.out.println("Producer %d waits".formatted(id));
54              try { wait(); }
55              catch (InterruptedException e) { Thread.currentThread().interrupt(); }
56          }
57          buffer[(firstIndex + size++) % bufferSize] = n;
58          System.out.println("Producer %d put: %d".formatted(id, n));
59          notifyAll();
60      }
61
62      public synchronized int get(int id) {
63          while (size == 0) {
64              System.out.println("Consumer %d waits".formatted(id));
65              try { wait(); }
66              catch (InterruptedException e) { Thread.currentThread().interrupt(); }
67          }
68          int value = buffer[firstIndex];
69          firstIndex = (firstIndex + 1 ) % bufferSize;
70          size--;
71          System.out.println("Consumer %d got: %d".formatted(id, value));
72          notifyAll();
73
74          return value;
75      }
76  }
77
78  public class PCWaitNotify {
79
80      private static void execute(int n1, int producerIterations, int n2, int
consumerIterations) {
81          Buffer buffer = new Buffer(100);
82          ExecutorService executorService = Executors.newFixedThreadPool(n1 + n2);
83          for (int i = 0; i < n1; i++) {
84              executorService.submit(new Producer(i, producerIterations, buffer));
85          }
86          for (int i = 0 ; i < n2; i++) {
87              executorService.submit(new Consumer(i, consumerIterations, buffer));
88          }
89          executorService.shutdown();
90      }
91
92      public static void main(String[] args) {
93  //          execute(1, 100, 1, 100);
94  //          execute(2, 100, 2, 100);
95  //          execute(3, 100, 1, 300);
96          execute(1, 300, 3, 100);
97      }
98  }
```

Fragment kodu 1: Plik `PCWaitNotify.java` - klasy `Producer`, `Consumer` oraz `PCWaitNotify`

3.2. Zadanie 1 – wykorzystanie semaforów

Klasy `Semaphore`, `BufferSemaphore` oraz `PCSemaphore` zaimplementowano w pliku `PCSemaphore.java` (fragment kodu 2). Klasy `Producer`, `Consumer` oraz `Buffer` są takie same jak w rozwiązaniu z metodami `wait()` oraz `notify()`. `BufferSemaphore` dziedziczy po `Buffer` i nadpisuje metody `put()` oraz `get()`.

`BufferSemaphore` wykorzystuje trzy semafory:

- `editSemaphore` (wartość początkowa 1) – semafor binarny pełniący rolę muteksu.
- `freeSemaphore` (wartość początkowa n = rozmiar bufora) – semafor licznikowy zliczający wolne miejsca.
- `fullSemaphore` (wartość początkowa 0) – semafor licznikowy zliczający zajęte miejsca.

Program uruchamiano dla konfiguracji $n_1 = n_2 = 1$, $n_1 = n_2 = 2$, $n_1 > n_2$ ($n_1 = 3$, $n_2 = 1$) oraz $n_1 < n_2$ ($n_1 = 1$, $n_2 = 3$). Liczby iteracji dobrano tak, aby $n_1 \cdot \text{liczba produkcji} = n_2 \cdot \text{liczba konsumpcji}$. We wszystkich przypadkach program zakończył się poprawnie, a zachowanie producentów i konsumentów było takie samo jak w rozwiązaniu z `wait()`/`notify()`.

```
1  package lab3;
2
3  import java.util.concurrent.ExecutorService;
4  import java.util.concurrent.Executors;
5
6  class Semaphore {
7      private int value = 0;
8
9      public Semaphore(int value) {
10         this.value = value;
11     }
12
13     public synchronized void P() {
14         while (value <= 0) {
15             try { wait(); }
16             catch (InterruptedException e) { Thread.currentThread().interrupt(); }
17         }
18         value--;
19     }
20
21     public synchronized void V() {
22         value++;
23         notify();
24     }
25 }
26
27 class BufferSemaphore extends Buffer {
28     Semaphore editSemaphore, freeSemaphore, fullSemaphore;
29
30     public BufferSemaphore(int n) {
31         super(n);
32         editSemaphore = new Semaphore(1);
33         freeSemaphore = new Semaphore(n);
34         fullSemaphore = new Semaphore(0);
35     }
```

```
36
37      @Override
38      public void put(int id, int n) {
39          freeSemaphore.P();
40          editSemaphore.P();
41
42          buffer[(firstIndex + size) % bufferSize] = n;
43          size++;
44          System.out.println("Producer %d put: %d".formatted(id, n));
45
46
47          fullSemaphore.V();
48          editSemaphore.V();
49      }
50
51      @Override
52      public int get(int id) {
53          fullSemaphore.P();
54          editSemaphore.P();
55
56          int value = buffer[firstIndex];
57          firstIndex = (firstIndex + 1 ) % bufferSize;
58          size--;
59          System.out.println("Consumer %d got: %d".formatted(id, value));
60
61          freeSemaphore.V();
62          editSemaphore.V();
63
64          return value;
65      }
66  }
67
68  public class PCSemaphore {
69      private static void execute(int n1, int producerIterations, int n2, int
consumerIterations) {
70          BufferSemaphore bufferSemaphore = new BufferSemaphore(100);
71          ExecutorService executorService = Executors.newFixedThreadPool(n1 + n2);
72          for (int i = 0; i < n1; i++) {
73              executorService.submit(new Producer(i, producerIterations,
bufferSemaphore));
74          }
75          for (int i = 0 ; i < n2; i++) {
76              executorService.submit(new Consumer(i, consumerIterations,
bufferSemaphore));
77          }
78          executorService.shutdown();
79      }
80
81      public static void main(String[] args) {
82  //          execute(1, 100, 1, 100);
83  //          execute(2, 100, 2, 100);
84  //          execute(3, 100, 1, 300);
85          execute(1, 300, 3, 100);
86      }
87  }
```

Fragment kodu 2: Plik `PCSemaphore.java` - klasy `Semaphore`, `BufferSemaphore` oraz `PCSemaphore`

3.3. Zadanie 2 – przetwarzanie potokowe

Klasy `Producer`, `Processor` oraz `Consumer` zaimplementowano w pliku `PipelineProcessing.java` (fragment kodu 3). Przyjęto bufor o rozmiarze 100, jednego producenta, jednego konsumenta i pięć uszeregowanych procesów przetwarzających. Wątkami zarządza `ExecutorService` z pulą o rozmiarze 7.

Wątki współdzielą bufor `buffer` oraz dwa rodzaje semaforów:

- `stage[0..5]` – `stage[i]` to semafor licznikowy zliczający elementy gotowe do obróbki przez i-ty etap
- `free` (początkowo 100) – semafor licznikowy zliczający wolne miejsca.

Prędkość przetwarzania w całym systemie zależy od jego najwolniejszego etapu. Rozmiar bufora decyduje, o ile producent może wyprzedzić resztę potoku.

Każdy wątek procesorów wykonuje tyle samo operacji co wszyscy producenci oraz konsumenci.

```
 1 package lab3;
 2
 3 import java.util.concurrent.ExecutorService;
 4 import java.util.concurrent.Executors;
 5 import java.util.concurrent.Semaphore;
 6
 7 public class PipelineProcessing {
 8     private static final int BUFFER_SIZE = 100;
 9     private static final int PRODUCER_COUNT = 1;
10     private static final int PRODUCER_ITERATIONS = 100;
11     private static final int PROCESSOR_COUNT = 5;
12     private static final int PROCESSOR_ITERATIONS = 100;
13     private static final int CONSUMER_COUNT = 1;
14     private static final int CONSUMER_ITERATIONS = 100;
15
16     private static final int[] buffer = new int[BUFFER_SIZE];
17     private static final Semaphore[] stage = new Semaphore[PROCESSOR_COUNT + 1];
18     private static final Semaphore free = new Semaphore(BUFFER_SIZE);
19
20     public static void main(String[] args) {
21         for (int i = 0; i < stage.length; i++) {
22             stage[i] = new Semaphore(0);
23         }
24         ExecutorService executorService =
25                 Executors.newFixedThreadPool(PRODUCER_COUNT + PROCESSOR_COUNT +
CONSUMER_COUNT);
26
27         for (int i = 0; i < PRODUCER_COUNT; i++) {
28             executorService.submit(new Producer(i, PRODUCER_ITERATIONS));
29         }
30         for (int i = 0; i < PROCESSOR_COUNT; i++) {
31             executorService.submit(new Processor(i, PROCESSOR_ITERATIONS));
32         }
33         for (int i = 0; i < CONSUMER_COUNT; i++) {
34             executorService.submit(new Consumer(i, CONSUMER_ITERATIONS));
35         }
36         executorService.shutdown();
37     }
```

```
38
39      static class Producer implements Runnable {
40          private final int id, iterations;
41
42          public Producer(int id, int iterations) {
43              this.id = id;
44              this.iterations = iterations;
45          }
46
47          public void run() {
48              try {
49                  for (int i = 0; i < iterations; i++) {
50                      free.acquire();
51                      int slot = i % BUFFER_SIZE;
52                      buffer[slot] = i;
53                      System.out.println("Producer %d put: %d at %d".formatted(id, i,
slot));
54                      stage[0].release();
55                      Thread.sleep((int) (Math.random() * 100));
56                  }
57              } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
58          }
59      }
60
61      static class Processor implements Runnable {
62          private final int id, iterations;
63
64          public Processor(int id, int iterations) {
65              this.id = id;
66              this.iterations = iterations;
67          }
68
69          public void run() {
70              try {
71                  for (int i = 0; i < iterations; i++) {
72                      stage[id].acquire();
73                      int slot = i % BUFFER_SIZE;
74                      buffer[slot] += 1;
75                      System.out.println("Processor %d processed: %d -> %d at
%d".formatted(id, i, buffer[slot], slot));
76                      stage[id + 1].release();
77                      Thread.sleep((int) (Math.random() * 100));
78                  }
79              } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
80          }
81      }
82
83      static class Consumer implements Runnable {
84          private final int id, iterations;
85
86          public Consumer(int id, int iterations) {
87              this.id = id;
88              this.iterations = iterations;
89          }
90
91          public void run() {
92              try {
93                  for (int i = 0; i < iterations; i++) {
94                      stage[PROCESSOR_COUNT].acquire();
95                      int slot = i % BUFFER_SIZE;
96                      System.out.println("Consumer %d got: %d from %d".formatted(id,
```

```
buffer[slot], slot));
 97                     free.release();
 98                     Thread.sleep((int) (Math.random() * 100));
 99                 }
100             } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
101         }
102     }
103 }
```

Fragment kodu 3: Plik `PipelineProcessing.java` - klasy `Producer`, `Processor` oraz `Consumer`

4. Wnioski

Oba rozwiązania problemu producenta i konsumenta, z `wait()`/`notify()` oraz z semaforami, zapewniają bezpieczeństwo oraz żywotność. Zachowanie obu wersji było takie samo dla wszystkich sprawdzonych konfiguracji n_1 i n_2.

Na jednym monitorze czekają zarówno producenci, jak i konsumenci. Pojedyncze `notify()` zamiast `notifyAll()` mogłoby obudzić wątek niewłaściwego typu, np. producenta po operacji producenta, i doprowadzić do zakleszczenia.

O przebiegu programu decyduje proporcja liczby producentów do liczby konsumentów.

W przetwarzaniu potokowym prędkość obróbki zależy od najwolniejszego etapu. Rozmiar bufora określa, o ile producent może wyprzedzić resztę potoku.